$\pi = [a_1, a_2, a_3]$ & $B = [h_1, \ldots$

Lab 12 - Decision Tree ~~Classifier~~ Regressor from scratch

Parameters of tree are Split dim, value child nodes

Sample	x_1	x_2	x_3	y
1	2	3	1	5
2	1	2	2	4
3	3	1	2	6
4	4	3	3	9
5	5	2	1	10
6	6	1	2	12

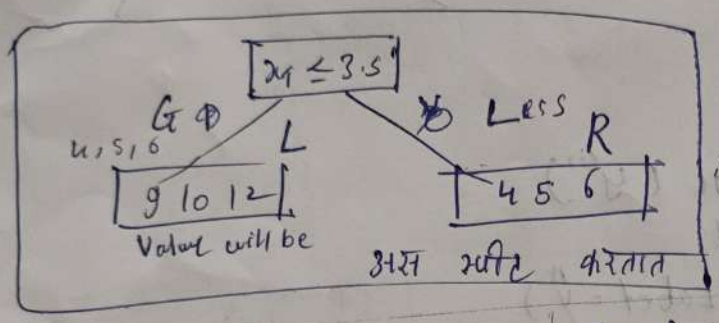
1) Take feature x_1
2) Sort feature values

$x_1 = [2, 1, 3, 4, 5, 6]$

$[1, 2, 3, 4, 5, 6]$

Candidate Split Values

$[1.5, 2.5, 3.5, 4.5, 5.5]$ → Split Data such that Y Values come with it



$x_1 = [1, 2, 3, 4, 5, 6]$
4 5 6 9 10 12

* Calculation of Loss & Cost is MSE

Predict Value for node is mean of all Samples at node

MSE_L Predict / Target Value

$$\hat{Y}_L = \frac{9+10+12}{3} \quad \frac{31}{3} = 10.33$$

$$\hat{Y}_R = \frac{4+5+6}{3} = 5$$

$$MSE_L = \frac{(4-5)^2 + (5-5)^2 + (6-5)^2}{3}$$

$$= \frac{1+0+1}{3} = 0.667$$

$$MSE_R = \frac{(9-10.33)^2 + (10-10.33)^2 + (12-10.33)^2}{3}$$

$$= 1.7689 + 0.1089 + 2.7889$$

$$= \boxed{1.556}$$

Total ERROR = $\boxed{2.222}$

The for every Split Value Calculate

For every feature for Split Value Calculate Total = Error & the the Lowest Split Value for feature

Selection → Lowest Split Value for feature

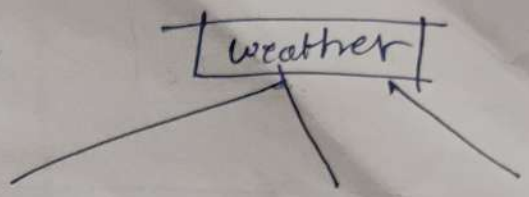
Feature	Candidate Split Value		Total (Error) Loss
x_1	1.5	→	6.64
	2.5	→	4.9375
	3.5	→	$\boxed{2.222}$
	4.5	→	4.5
	5.3	→	4.14
x_2	1.5	→	
	2.5	→	
x_3	1.5	→	
	2.5	→	

product of 2 vector

k 2 = if leaf certain less tuss than equal to 2 leaf node Stop (build tress) সেখানে হয়ের পেরেন্টার হিসাবে দেখা যাবে।

```
BuidTree (I, k)
{
  k = No. of Sample to Stop = 2
  I = list of Sample
```

k = No of Sample
I = No of list of nodes




```
Build Tree (I, k)
{
  if (|I| ≤ k)
    {
      Set ŷ = average (y^(i))
                  i∈I
      return Leaf (Label = ŷ)
    }
  else
    {
      For each Split dimension j
      {   for each Split Values s
          {
            I+_{j,s} = { i ∈ I | x_j^(i) ≥ s }
            I-_{j,s} = { i ∈ I | x_j^(i) ≤ s }
            ŷ+_{j,s} = average (y^(i))
                       [i ∈ I_{js}]
            ŷ-_{js} = average (y^(i))
                      [i ∈ I_{js}]
            E_{js} = Σ_{i∈I+_{js}} (y^(i) - ŷ+_{js})^2 + Σ_{i∈I-_{js}} (y^(i) - ŷ-_{js})^2
                                                        → append E_{js}
          } # for each values
      }
      = argmin E_{j,s}, j, s = j* < s*
  return Node, (j*, s*) Buildtree, I-_{j*s*}, k),
                        Build Tree (I+_{j*s*}, k)
```